

Digital Humanities Data Mining & Visualization

Client/Mentor

Hallie G. Meredith, D.Phil.

Dr. Parteek Kumar

The Historians

Jefferson Kline

Luis Garcia Rangel

Nicholas Vendeland

Project Overview

- Artifact Webapp MVP is a full-stack web app for managing historical artifact records.
- Users can sign in, create and manage artifact records, organize them into collections, search/browse artifacts, and view detailed artifact information.
- The goal is to make artifact documentation easier, cleaner, and more accessible through a centralized web platform.

Problem Statement

- Artifact information is often scattered, inconsistent, or stored manually across files, spreadsheets, and external databases.
- This makes it hard to search, organize, update, and display artifact records in a consistent way.
- The project solves this by providing one structured system for storing, viewing, and managing artifact data.

Project Scope

- Build an MVP web application for artifact record management.
- Include user authentication, artifact CRUD functionality, collection organization, search/browse views, and database-backed storage.
- Support future expansion for external data imports and larger museum/archive workflows.
- Target users include students, researchers, collectors, museums, archives, and anyone managing artifact metadata.

System/Implementation Requirements

- Secure user authentication and user-specific artifact management.
- Full artifact and collection functionality: create, view, update, delete, browse, search, and organize records.
- Connected full-stack structure with frontend pages, backend API routes, and PostgreSQL database models using consistent data fields.
- Built the React frontend with pages for browsing, profiles, collections, records, and reusable UI components.
- Built the Express/TypeScript backend with API routes and controllers for users, records, and collections.
- Added Clerk authentication, Drizzle/PostgreSQL database models, structured artifact metadata, and future import support.

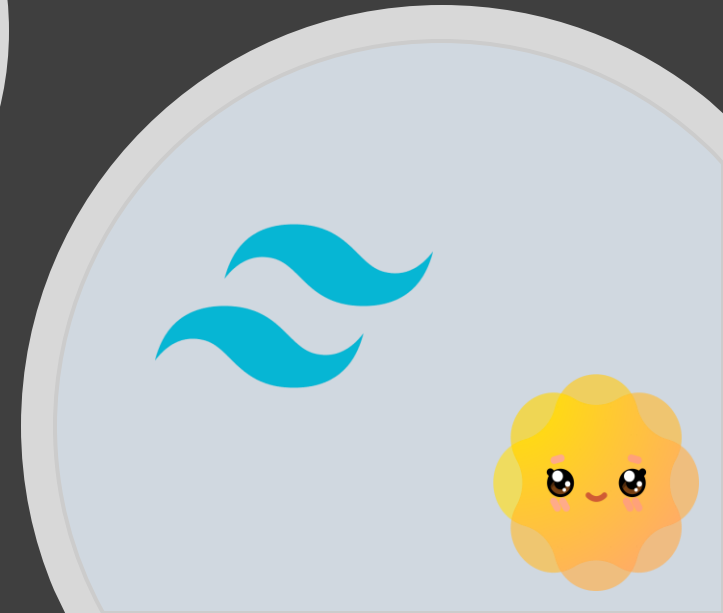
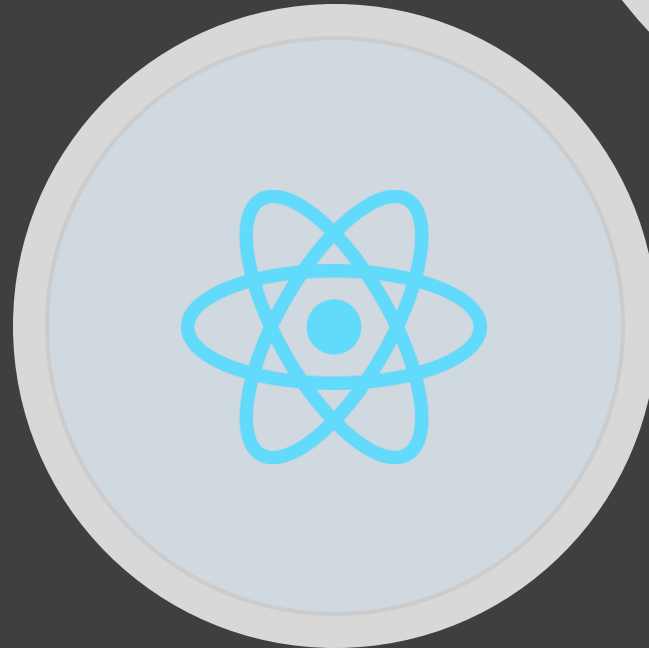
Technologies Used

Architecture: Frontend

Structure: React + Vite

Styling: Tailwind + DaisyUI + Lucide Icon Library

Data Fetching/Caching: Tanstack Query

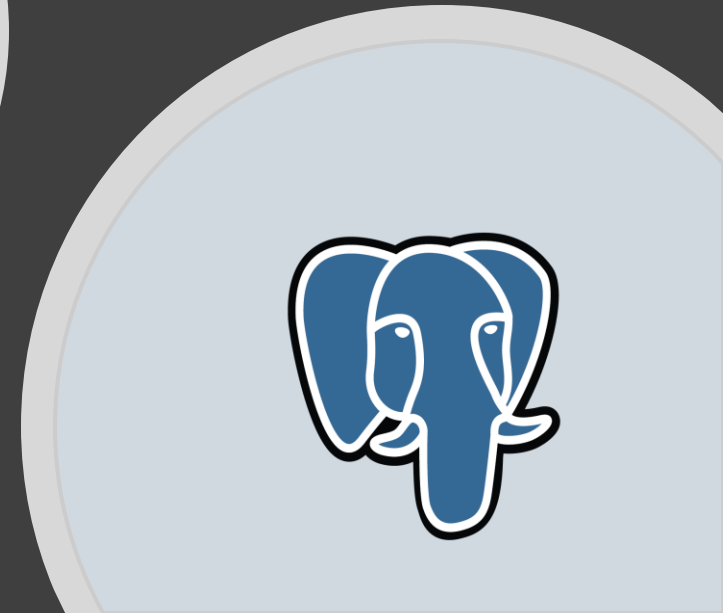


Architecture: Backend

Structure + API: Express.js +
Node

Authentication: Clerk

Database: PostgreSQL Hosted
on Neon + Drizzle ORM



System Architecture Overview

Website Client Server Architecture

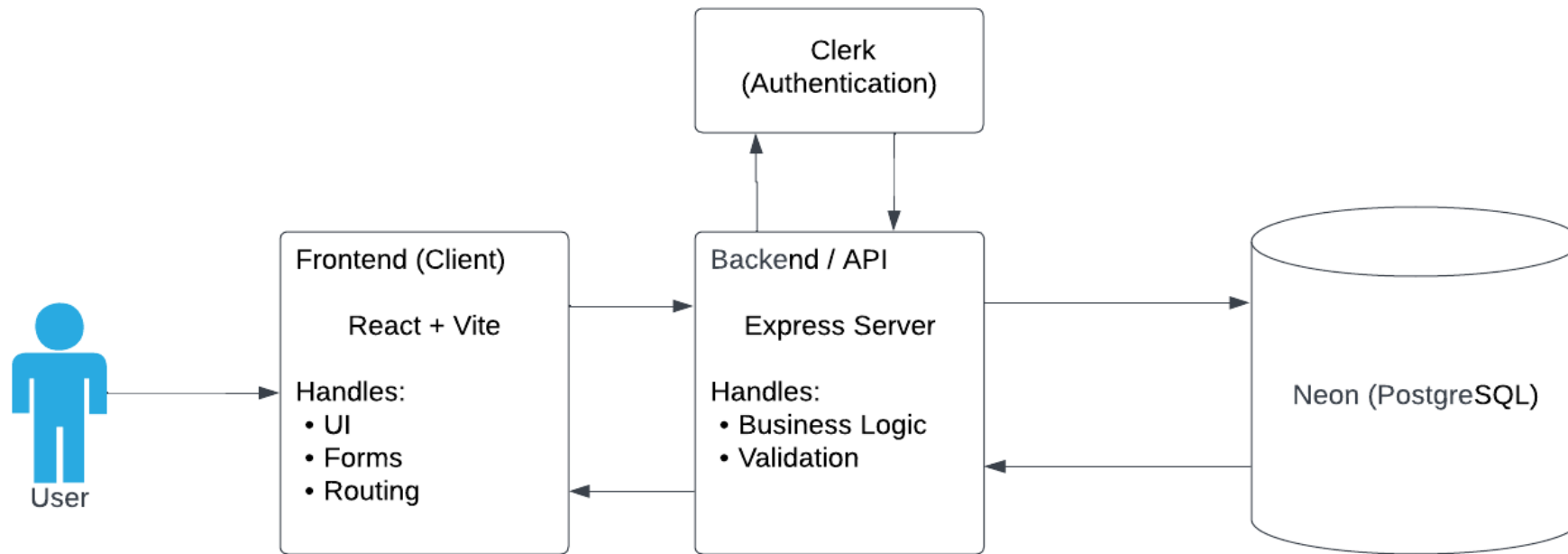
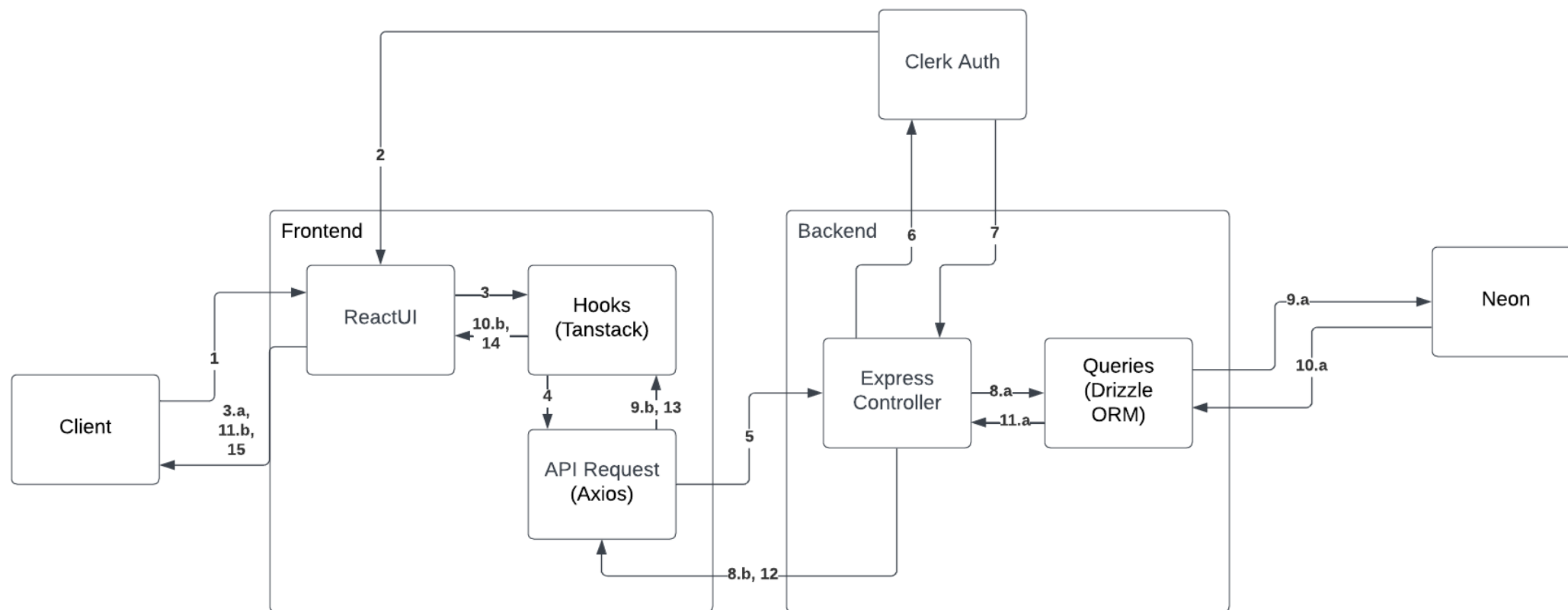


Diagram and Component Interaction

Architecture Diagram and Component Interaction (Record Creation)





Implemented Features:

Guest View

Login/Register

User View

User Auth

Profile Viewing

Record Creation

Record Editing

Record Deletion

[Begin Demo]

Sprint 3 Achievements

- Milestones Reached:
 - Functional Web application for the client to use
 - Functional Data pipeline
- Features Improved:
 - Faster querying
 - Functional frontend from prototype
 - Proper data parsing through data pipeline form large data sets

Sprint 3: Contributions

Luis

- Frontend
 - Migrated EJS prototype templates to react components
 - Build page components
 - Implemented Protected Routes
 - Frontend/Backend integration with Axios + Tanstack
- Backend
 - Record creation workflow end-to-end
 - Rest API endpoints for records, users, and collections
 - Authentication with Clerk
- Database
 - User sync between Clerk and Neon
 - Contributed to database Schema design
- System Architecture & Data flow

Sprint 3: Contributions (Nick)

- Data Ingestion

- EDH (Heidelberg) ingestion prototype
- ETL scaffolding for additional sources
- Schema mapping for messy external metadata
- Field normalization and validation logic
- Logging unmapped fields instead of silently dropping

Sprint 3 Retrospective

- **What went well:**

- Schema settled early and held up through the whole sprint; Drizzle + Neon was painless to iterate on.

- **Challenge:**

- External epigraphic data is messy, inconsistent fields, multilingual encoding, optional metadata. Mapping it cleanly without losing meaning was hard.

- **Solution:**

- Built ingestion as a separate ETL stage so the live app stayed clean. Used the typed schema as a contract, unmapped fields get logged, not silently dropped.

- **Team:**

- Communication on schema decisions was solid; standups kept frontend, backend, and database aligned on field names and shapes.

Testing Strategy

- What went well:

- Schema settled early and held up through the whole sprint; Drizzle + Neon was painless to iterate on.

- Challenge:

- External epigraphic data is messy, inconsistent fields, multilingual encoding, optional metadata. Mapping it cleanly without losing meaning was hard.

- Solution:

- Built ingestion as a separate ETL stage so the live app stayed clean. Used the typed schema as a contract, unmapped fields get logged, not silently dropped.

- Team:

- Communication on schema decisions was solid; standups kept frontend, backend, and database aligned on field names and shapes.

AI Usage

- Minor bug fixes/drafting, but everything was revised into our own voice.
- No AI generated content is committed without a team member reviewing it; final decisions on architecture and design are with us.

Future Plans & Deliverables

- Finish the placeholders: complete EditRecordPage, CollectionPage, and the SearchPage routing so every nav link lands on a working view.
- Native image upload: replace the current imageUrl text field with file uploads to a hosted object store, with thumbnails on record cards.
- External database ingestion: wire the EDH prototype into the live app and add additional sources (IRT, I.Sicily, OCRE) as ETL scripts.
- Edit history and multi-contributor attribution so credit is preserved when records are modified by other users.
- Automated test coverage and a basic CI pipeline so regressions get caught before they reach the client.

Timeline for Next Phase

- Weeks 1–2: Cleanup sprint. Fix outstanding bugs (Navigate import, route mismatches, collection endpoint issues) and stabilize the existing CRUD paths.
- Weeks 3–5: Native image upload + EDH ingestion wired to the live app. First showcase-ready external data flowing in.
- Weeks 6–8: Additional ingestion sources (IRT, I.Sicily, OCRE/Nomisma) and edit history / attribution schema.
- Weeks 9–10: Automated tests, basic CI, and polish pass. Final MVP showcase to the client.

Conclusion & Key Takeaways

- We have a working full-stack MVP: React frontend, Express/TypeScript backend, Clerk auth, and a PostgreSQL database the client can actually use.
- Sprint 3 moved us from prototype to functional product, record CRUD, user sync, and structured artifact metadata are all live.
- Known gaps are tracked honestly (placeholder pages, no native uploads, no edit history yet) and slotted into next sprint.
- Strongest priority going forward: external database ingestion, starting with EDH, to make the platform genuinely useful to researchers.
- The architecture (Drizzle + Neon + Clerk + React) is holding up well and gives us room to scale features without rewrites.